write clean, predictable, and modular code. Expect questions not only about defining concepts like pure functions, immutability, and higher-order functions, but also about applying them effectively in real-world scenarios.

Candidates should feel confident explaining and implementing `map`, `filter`, and `reduce`, refactoring imperative code into functional pipelines, and discussing trade-offs when blending FP with other paradigms like Object-Oriented Programming (OOP). For advanced interviews, be prepared to tackle topics like currying, partial application, and optimizing functional flows for performance and scalability.

Table 3.2 provides Jane's review guide for functional paradigms. Use it as a checklist to ensure you are prepared to tackle both conceptual and practical challenges with confidence.

Table 3.2 Core concepts of functional paradigms

Topic	Fundamentals	Applied skills	Advanced
Functional programming	Identify pure functions; use `map`, `filter`, `reduce`; explain immutability.	Refactor code for immutability; apply currying and partial application; use higher-order functions (HOF).	Architect functional modules; optimize performance in functional pipelines; blend FP with OOP patterns.
Iterators & generators	Use `for...of`; explain iterators and iterable protocols.	Create custom iterators; use `yield` to build generator functions.	Build asynchronous flows with generators; integrate iterables in advanced design patterns.
Symbols & meta-programming	Explain `Symbol` basics and well-known symbols (	Use `Proxy` for validation, intercept API	Architect meta-programming utilities for